

Documentación de Script: Extractor y Buscador de Cuentas en Logs (FNA)

Objetivo General

Optimizar el rastreo de transacciones mediante la automatización de la búsqueda de cuentas específicas dentro de los archivos planos de log (`PAG\_\*.TXT`). El script realiza el filtrado por fecha e inspecciona eficientemente la cabecera (primera línea) de múltiples archivos simultáneamente desde la consola del navegador, evitando descargas manuales.

📄 Instrucciones de Uso

1. Preparación

1. Navega a la ruta del servidor de logs en tu navegador:

https://applogs.ecollect.co/applogs/Mod_Admin/FNA/

2. Abre las herramientas de desarrollador presionando F12 (o clic derecho -> Inspeccionar) y dirígete a la pestaña Consola.

3. Copia el script completo que esta en script_log_FNA-minificado.txt, pégalo en la consola y presiona Enter para cargar la función `buscar()`.

> 💡 Nota: Si es la primera vez que usas la consola, el navegador podría solicitar escribir la frase `allow pasting` o similar para lograr pegar el script.

2. Sintaxis de Ejecución

A. Búsqueda Básica (Una o más cuentas)

```
// ejecuta la funcion de la forma:  
buscar("cuenta1, cuenta2", "mes/día/año");
```

⚠️ La fecha debe estar escrita tal cual esta en la pagina de log.

~~~javascript

```
//Ejemplo:  
buscar('18993758', '6/30/2026')
```

```
//Ejemplo varias cuentas a la vez:  
buscar('33791187, 33395567 ', '6/30/2026')
```

~~~

B. Búsqueda Avanzada (Excluyendo archivos específicos -- opcional)

El tercer parámetro te permite ignorar archivos de ejecuciones previas o duplicadas para no sesgar el resultado. Puedes ponerlos con o sin la extensión ``.txt``.

```
// ejecuta la funcion de la forma:
buscar("cuenta1, cuenta2", "mes/día/año", "archivo_a_ignorar1,
archivo_a_ignorar2");
```

~~~javascript

```
//Ejemplo:
buscar('33791187, 33395567', '6/30/2026', 'PAG_10020_20260626_094213.txt,
PAG_10020_20260626_094439 ')
```

~~~

🗨 Lógica del Script

El script ejecuta un proceso secuencial dividido en cuatro etapas principales:

1. Validación de Entradas:

Comprueba que los formatos de las cuentas sean correctos (solo números, comas o espacios) y que se haya ingresado una fecha de referencia.

2. Filtrado del DOM (Página Web):

Localiza todos los enlaces de archivos que contengan la nomenclatura ``/PAG_``.

* Analiza el nodo de texto inmediatamente anterior a cada enlace para validar si coincide con la fecha de interés.

* (Opcional) Excluye los archivos parametrizados en la lista de ignorados y los visualiza en color gris.

3. Valida el contenido de cada archivo filtrado de forma asíncrona. Para optimizar el rendimiento solo inspecciona la primera línea (fila de cabecera) de cada archivo plano para buscar las coincidencias de las cuentas.

4. Inyección Visual y Resumen:

* Encontrado: Pinta el enlace en verde, añade un borde destacado e inyecta dinámicamente el número de cuenta hallado al lado del enlace (🔍 #Cuenta).

* Genera un objeto JSON final que se imprime en la consola del desarrollador y también se renderiza en una ventana flotante (``#resumen-busqueda``).

* ⚠ Script solo valida si la cuenta esta en dicho archivo, es responsabilidad del funcionario:

- validar numero cuenta de forma correcta en el archivo.
- validar fecha descarga.
- validar registro (tamaño de la linea).
- validar numero de registros.
- validar el saldo (en la ultima linea)
- descargar LOG y compartir como 'Avances y Novedades:'

📄 Información del Script y Control de Cambios

****Autor**** Alvaro Ruiz

****Última Modificación**** 03/07/2026

****Versión**** v260703

****Entorno de Ejecución**** Consola del Navegador (DevTools)

****Tecnologías**** JavaScript (Async/Await, QuerySelectors, Fetch API) y CSS